

Tecnología Digital VI

Procesamiento de texto y RNNs

Licenciatura en Tecnología Digital - UTDT

NLP y otras secuencias

En las últimas clases le dimos un tratamiento particular a las imágenes. Vimos redes específicas para diversos problemas sobre ellas.

Podemos hacer un camino similar con el tratamiento de *secuencias de datos*.

Contextos de aplicación específicos de *secuencias de datos*:

- Procesamiento de lenguaje natural: texto, habla, etc.
- Procesamiento de audio. En el mundo de las señales está junto con las imágenes, en el mundo de las NNs más pegado al texto.
- Procesamiento de video. Un poco de secuencias, un poco de imágenes.
- Series de tiempo.
- Proteínas.

Hoy **EL problema estrella del Procesamiento del Lenguaje Natural son los Language Models.** Sin embargo hay diversos problemas sobre texto que tienen tratamiento y buenos resultados mucho antes que los Language Models tuvieran un éxito rotundo.

Algunos ejemplos sintácticos, semánticos y generales:

- POS (Part-of-Speech) tagging: Etiquetar las palabras como sustantivos, verbos, adjetivos, etc.
- NER (Named Entity Recognition): Reconocer sustantivos propios como personas, entidades, lugares.
- Sentiment Analysis.
- Noun-Compound Interpretation.
- Topic Modelling.
- Text Summarization.

Noun-Compound Interpretation

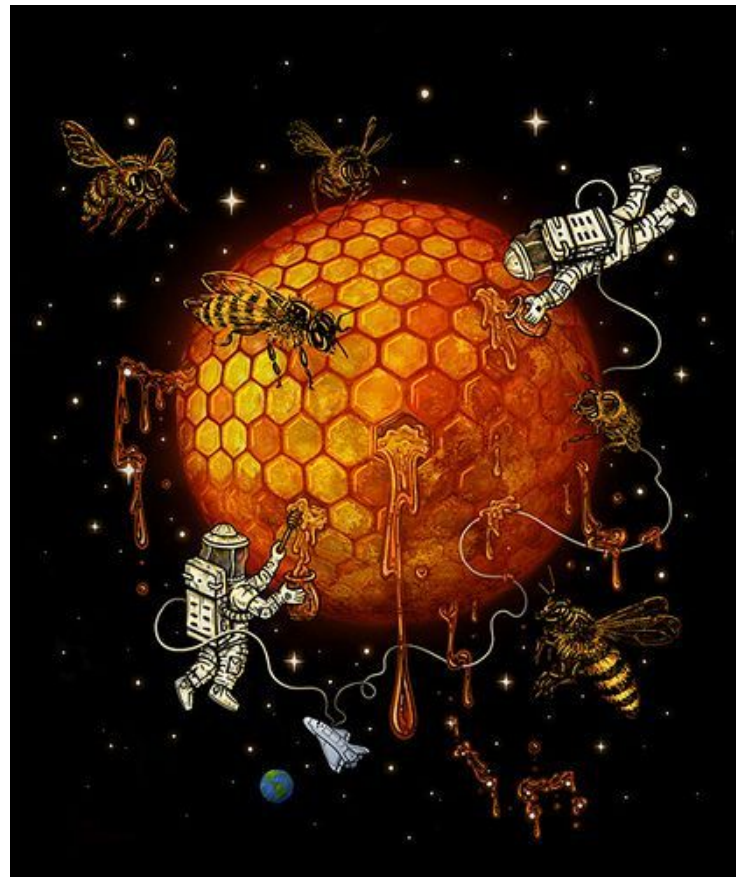
No todo son Language Models y generar la siguiente palabra.

Un ejemplo: los sustantivos compuestos son sustantivos que se forman componiendo dos palabras (en general sustantivos). Guardian entre ellos una relación semántica no explícita.

Oil pump: una bomba que bombea aceite.

Aluminium pump: una bomba hecha de aluminio.

Es un problema de clasificación (con unas 45-50 categorías) que se trata desde la era *pre-neural*.



Dificultades del texto

El texto y el lenguaje en general presentan ciertas dificultades por sobre las imágenes.

Lo que hay en una imagen tiene diferencias alrededor del mundo en su semántica (símbolos, costumbres, etc), pero estructuralmente una imagen es igual en Argentina y en cualquier otro país.

Con el lenguaje las diferencias son directamente estructurales además de semánticas.

Dificultades del texto

- Diferentes símbolos.
- Se tokenizan distinto.
- Se leen en diferente dirección.
- Agregan características en el habla como la tonalidad.
- Lenguajes aislados.

LA VIDA ES SUEÑO

Jornada III
Escena XIX

SEGISMUNDO

Es verdad, pues: reprimamos esta fiera condición, esta furia, esta ambición, por si alguna vez soñamos. Y si haremos, pues estamos en mundo tan singular, que el vivir sólo es soñar; y la experiencia me enseña, que el hombre que vive, sueña lo que es, hasta despertar.

Sueña el rey que es rey, y vive con este engaño mandando, disponiendo y gobernando; y este aplauso, que recibe prestado, en el viento escribe y en cenizas le convierte la muerte (¡desdicha fuerte!); ¡que hay quien intente reinar viendo que ha de despertar en el sueño de la muerte!

Sueña el rico en su riqueza,

que más cuidados le ofrece; sueña el pobre que padece su miseria y su pobreza; sueña el que a medrar empieza,

sueña el que afana y pretende, sueña el que agravia y ofende, y en el mundo, en conclusión, todos sueñan lo que son, aunque ninguno lo entiende.

Yo sueño que estoy aquí, destas prisiones cargado; y soñé que en otro estado más lisonjero me vi. ¿Qué es la vida? Un frenesí. ¿Qué es la vida? Una ilusión, una sombra, una ficción, y el mayor bien es pequeño; que toda la vida es sueño, y los sueños, sueños son.

שׁוֹשֶׁנָה יַעֲקֹב צִהְלָה וְשִׁמְחָה, בְּרֹאשׁוֹם יְחִיד תְּכַלֵּת מִדְּרָכֶי, תְּשׁוּעָתָם הָיִית לְנִצָּח, וְתִקְוָתָם בְּכָל דּוֹר וָדוֹר. לְהוֹדִיעַ שָׁכָל קִוְיָךְ לֹא יִבְשׁוּ וְלֹא יִכְלְמוּ לְנִצָּח כָּל הַחוֹסִים בְּךָ. אֲדוֹר הַמֶּן אֲשֶׁר בִּקֵּשׁ לְאֲבֹדִי, בְּרוּךְ מִדְּרָכֵי הַיְּהוּדִי. אֲדוֹרָה וְרֵשׁ אִשֶּׁת מִפְּחָדִי, בְּרוּכָה אֶסְתֵּר בַּעֲדִי, אֲדוֹרִים כָּל הָרָשָׁעִים, בְּרוּכִים כָּל הַצְדִּיקִים, וְגַם תְּרַבּוּנָה זְכוֹר לְטוֹב:

如果别人跟小木偶说要等上一天而不是二十分钟，那时间对于他说简直不能更难熬了，他心急如焚地来回踱步，走到这头又折回头。终于，他一个转身，鼻子正朝着奇迹之地的方向。他一边脚步匆忙地赶路，一边听到自己激动的心跳，嘞，咚，嘞，咚，跟壁钟似的，同时小脑袋不断地在想：万一，我种出来的不是一千个，而是两千个呢？或者万一，我种出来的不是两千个，而是五千个 - 甚至至是十万个呢？那我就要给自己建一座华美的宫殿有一千个马廐，装满一千匹木马给我玩，还有一个盛满了柠檬水和冰淇淋苏打的地窖，还有一间装满蜡烛和水果，蛋糕和饼干的图书馆。

من نوادر جحا

-1-

كان جحا راكبا حماره حينما مر ببعض القوم وإراد احدهم ان يمزح معه فقال له : يا جحا لقد عرفت حمارك ولم اعرفك فقال جحا : هذا طبيعي لان الحمير تعرف بعضها.

-2-

مشى جحا في طريق، فدخلت في رجله شوكه فآلمته، فلما ذهب إلى بيته أخرجهما وقال : الحمد لله، فقالت زوجته : على أي شيء تحمد الله؟ قال : أحمدته على أنني لم أكن لابساً حذائي الجديد وإلا خرقتة الشوكه.

-3-

جاء رجل لجحا يريد ان يصف له علاجاً للسعال فقال له جحا : عليك بشرب كوباً من حليب الحمار وفي اليوم الثاني سال جحا الرجل فقال : كيف حالك اليوم فقال الرجل احسن من الامس فقال جحا : عامل ولدنا معامله حسنة لانه اليوم صار اخوك من الرضاعة.

La regla OSASCOMP indica cómo es el orden los adjetivos en inglés. La gente la usa sin tener idea de que lo está haciendo.

A large Italian pizza.

An Italian large pizza.

Los adjetivos *van* en el orden
opinion-size-age-shape-color-origin-material-purpose.

She found a valuable, small, antique, rounded, brown, Italian, leather, travel bag.

She found a leather, brown, travel, rounded, antique, Italian, valuable, small bag.

Sumando dificultades

Vimos que estructuralmente el lenguaje es muy distinto a las imágenes.

Adicionalmente, si nuestra idea es resolver problemas de lenguaje utilizando Machine Learning y Deep Learning, se nos suma una dificultad más.

El texto no se traduce trivialmente a un número. Nuestra forma computacional y matemática de representar imágenes son directamente números. Nuestra forma de representar caracteres sí es numérica (computacionalmente todo lo es), pero no tenemos una representación trivial numérica de las palabras o de los *tokens** (la concatenación de caracteres no va a ser una gran idea).

NLP pre-neural

En la era pre-neural igualmente se utilizaban métodos de Machine Learning que esperaban features numéricos.

Bag-of-words model | Ilustración

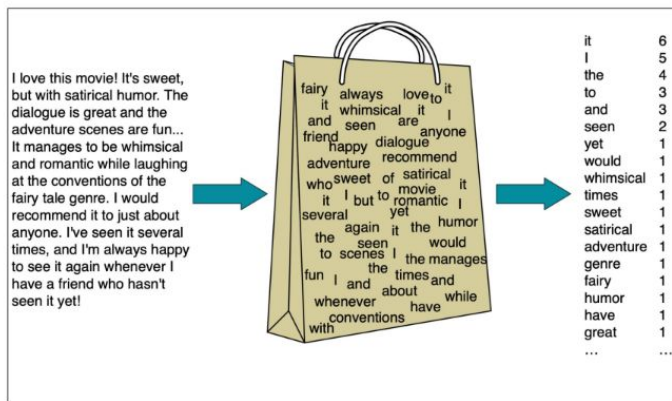


Figure 4.1 Intuition of the multinomial naive Bayes classifier applied to a movie review. The position of the words is ignored (the *bag-of-words* assumption) and we make use of the frequency of each word.

Fuente: "Naive Bayes and Sentiment Classification" en Jurafsky, D., & Martin, J. H. (2023). Speech and Language Processing. Disponible [aquí](#).

Bag-of-words model | Matriz documento-término

La matriz documento-término (**DTM**, por las siglas en inglés: *document-term matrix*) tiene:

- Tantas **filas** como **documentos** distintos tenga el corpus.
- Tantas **columnas** como **tokens** distintos tenga el corpus.

	battle	good	fool	wit
As You Like It	1	114	36	20
Twelfth Night	0	80	58	15
Julius Caesar	7	62	1	2
Henry V	13	89	4	3

A la DTM se la puede usar para entrenar modelos.

Bag-of-words model | Algunos de sus problemas

- Si el vocabulario es grande, hay **muchas columnas**.
- Perdemos toda la información referida al **orden** de las palabras. ¿`toy dog` (perro de juguete) = `dog toy` (juguete para perros)?
- Ignora el **contexto** de las palabras, como el que estén **negadas**. ¿`La comida era rica` = `La comida no era rica`?
- Ignora la **similitud semántica** entre palabras. En consecuencia, palabras que significan lo mismo pueden terminar en columnas distintas. ¿`Auto` != `Automóvil`? ¿`Encendedor` != `Mechero`?
- No contempla la **polisemia** (i.e., pluralidad de significados de una expresión lingüística; [fuente](#)).

Normalización TF-IDF

Es el **producto de 2 términos**:

- **Frecuencia de la palabra (TF)**. La frecuencia de la palabra t en el documento d .
 - **Le quita peso a una palabra excesivamente frecuente en un documento.**
 - El que una palabra aparezca 100 veces en un documento no hace que esa palabra sea 100 veces más relevante para el significado del documento.

$$\text{tf}_{t,d} = \log_{10}(\text{count}(t,d) + 1)$$

Fuente: "Vector Semantics and Embeddings" en Jurafsky, D., & Martin, J. H. (2023). Speech and Language Processing. Disponible [acá](#).

Normalización TF-IDF (continuación)

- **Frecuencia inversa del documento** o peso idf de la palabra (**IDF**). N es el número total de documentos en el corpus. df_t es el número de documentos en que ocurre la palabra t .
 - **Le quita peso a una palabra que aparece en todos los documentos.**
 - En otras palabras, se incorpora para dar mayor peso a palabras que ocurren sólo en unos pocos documentos, ya que son más útiles para discriminar esos documentos del resto, relativo a términos que ocurren frecuentemente a lo largo de todo el corpus.
 - A menor cantidad de documentos en que esta palabra ocurre, mayor es este peso.

$$idf_t = \log_{10} \left(\frac{N}{df_t} \right)$$

Fuente: "Vector Semantics and Embeddings" en Jurafsky, D., & Martin, J. H. (2023). Speech and Language Processing. Disponible [acá](#).

Normalización TF-IDF (continuación)

Entonces, el producto de estos 2 términos le da **mayor importancia** a las palabras que **aparecen muchas veces y en pocos documentos**.

$$w_{t,d} = \text{tf}_{t,d} \times \text{idf}_t$$

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Figure 6.2 The term-document matrix for four words in four Shakespeare plays. Each cell contains the number of times the (row) word occurs in the (column) document.

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	0.074	0	0.22	0.28
good	0	0	0	0
fool	0.019	0.021	0.0036	0.0083
wit	0.049	0.044	0.018	0.022

Figure 6.9 A tf-idf weighted term-document matrix for four words in four Shakespeare plays, using the counts in Fig. 6.2. For example the 0.049 value for *wit* in *As You Like It* is the product of $\text{tf} = \log_{10}(20 + 1) = 1.322$ and $\text{idf} = .037$. Note that the idf weighting has eliminated the importance of the ubiquitous word *good* and vastly reduced the impact of the almost-ubiquitous word *fool*.

Fuente: "Vector Semantics and Embeddings" en Jurafsky, D., & Martin, J. H. (2023). Speech and Language Processing. Disponible [aquí](#).

One-Hot Encoding

Una posible opción para **tratar a las palabras como números** es aplicar la misma metodología de variables categóricas que hicimos en ML clásico. **Cada palabra o token es su propia categoría.**

El problema de esta representación es que estamos llevando las palabras a un espacio que carece completamente de semántica.

- El **espacio tiene tanta dimensiones como palabras** tenga tu lenguaje.
- El producto interno, muchas veces utilizado para definir la distancia coseno de dos vectores (su similitud), es 0 para todo par de palabras.
- **Todo es equidistante.** La distancia de Perro a Lobo es la misma que la de Perro a Mariposa.

Label encoding

Otra opción es el Label encoding. Traducimos cada palabra a un número entre 1 y n , siendo n la cantidad de palabras en el lenguaje.

Si bien es una representación más simple que OHE no solo que todavía no tenemos ninguna semántica asociada a esos números sino que incluso introducimos semántica completamente ficticia.

Si Perro es 1, Mariposa es 2 y Lobo es 3, ahora la distancia entre Perro y Lobo es el doble que la de Perro a Mariposa.

Históricamente hubo varios esfuerzos de llevar las palabras a un espacio con semántica interesante. Uno de los resultados más exitosos en esta dirección fue Word2Vec.

La idea principal de Word2Vec es que podemos caracterizar a una palabra por las palabras que suelen aparecer a su alrededor.

Efficient Estimation of Word Representations in Vector Space

Tomas Mikolov

Google Inc., Mountain View, CA

tmikolov@google.com

Kai Chen

Google Inc., Mountain View, CA

kaichen@google.com

Greg Corrado

Google Inc., Mountain View, CA

gcorrado@google.com

Jeffrey Dean

Google Inc., Mountain View, CA

jeff@google.com

Abstract

We propose two novel model architectures for computing continuous vector representations of words from very large data sets. The quality of these representations is measured in a word similarity task, and the results are compared to the previously best performing techniques based on different types of neural networks. We observe large improvements in accuracy at much lower computational cost, i.e. it takes less than a day to learn high quality word vectors from a 1.6 billion words data set. Furthermore, we show that these vectors provide state-of-the-art performance on our test set for measuring syntactic and semantic word similarities.

Word2Vec

El objetivo de Word2Vec era **lograr un *word embedding* interesante**. Es decir, poder **proyectar las palabras en vectores de un cierto espacio en donde se cumplan *propiedades interesantes*** que luego faciliten la resolución de problemas.

Word2Vec en realidad no es un modelo, son 2. El modelo CBOW y el modelo Skip-Gram.

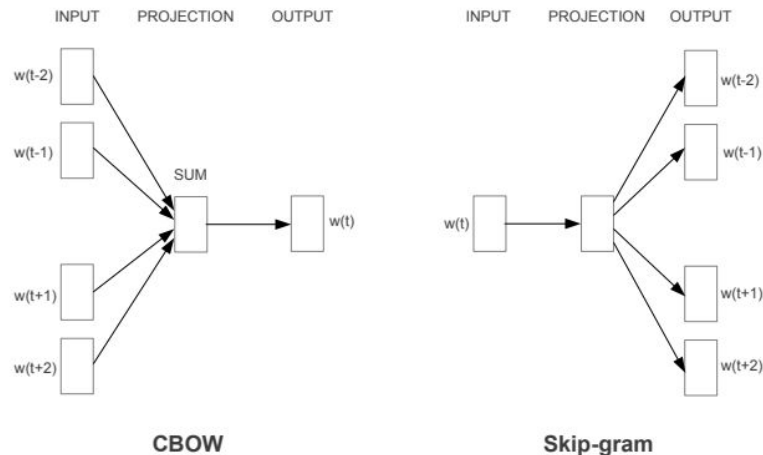


Figure 1: New model architectures. The CBOW architecture predicts the current word based on the context, and the Skip-gram predicts surrounding words given the current word.

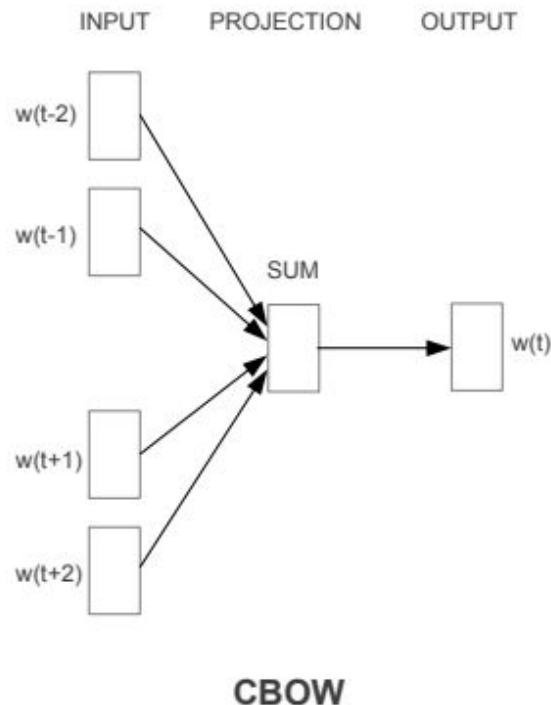
La idea de CBOW es poder predecir la palabra central de una frase en base a las palabras que están antes y después.

3.1 Continuous Bag-of-Words Model

The first proposed architecture is similar to the feedforward NNLM, where the non-linear hidden layer is removed and the projection layer is shared for all words (not just the projection matrix); thus, all words get projected into the same position (their vectors are averaged). We call this architecture a bag-of-words model as the order of words in the history does not influence the projection. Furthermore, we also use words from the future; we have obtained the best performance on the task introduced in the next section by building a log-linear classifier with four future and four history words at the input, where the training criterion is to correctly classify the current (middle) word. Training complexity is then

$$Q = N \times D + D \times \log_2(V). \quad (4)$$

We denote this model further as CBOW, as unlike standard bag-of-words model, it uses continuous distributed representation of the context. The model architecture is shown at Figure 1. Note that the weight matrix between the input and the projection layer is shared for all word positions in the same way as in the NNLM.

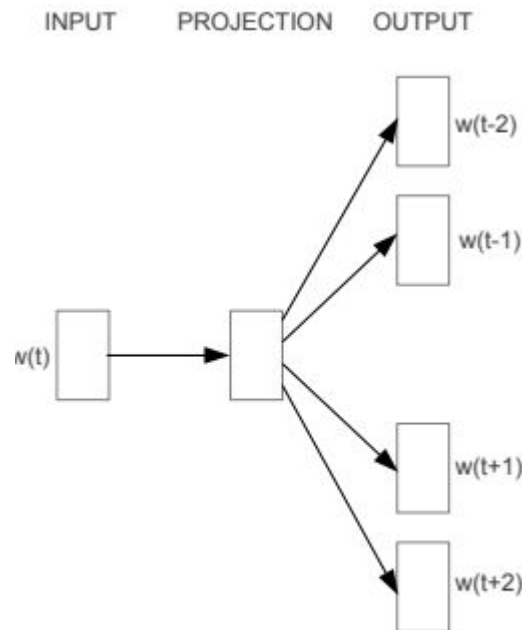


Skip-Gram

La idea de Skip-Gram es a partir de una palabra central, predecir las palabras anteriores y posteriores en una frase.

3.2 Continuous Skip-gram Model

The second architecture is similar to CBOW, but instead of predicting the current word based on the context, it tries to maximize classification of a word based on another word in the same sentence. More precisely, we use each current word as an input to a log-linear classifier with continuous projection layer, and predict words within a certain range before and after the current word. We found that increasing the range improves quality of the resulting word vectors, but it also increases the computational complexity. Since the more distant words are usually less related to the current word than those close to it, we give less weight to the distant words by sampling less from those words in our training examples.



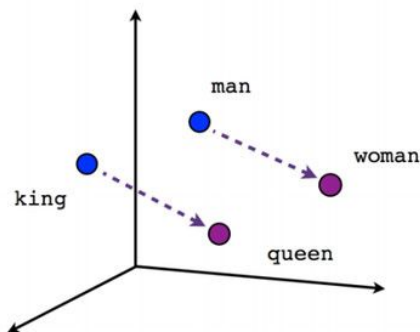
Skip-gram

El entrenamiento del modelo CBOW lo podemos enumerar coloquialmente en los siguientes pasos:

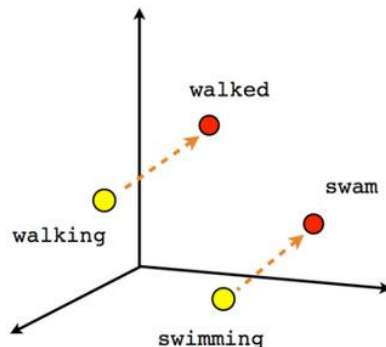
- En una frase se toman las palabras previas y posteriores a la central.
- Se pasa cada palabra a su representación OHE.
- Se las multiplica por una matriz W que te lleva a un espacio latente denso (mucho menor dimensión que el de OHE).
- Se junta las palabras promediándolas (no importa el orden -> CBOW).
- Se pasa el nuevo vector por una matriz W' para proyectarlo a un vector de *logits* de dimensión igual al de OHE.
- Se aplica softmax para convertir los logits en probabilidades.
- Se compara contra la *ground truth* de cuál era la palabra real y se actualiza W y W' en consecuencia usando la *loss functions*.

Word2Vec embedding

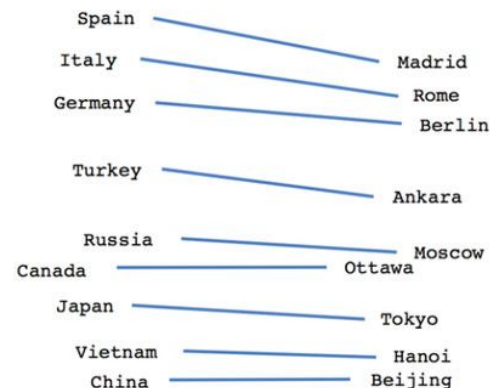
Una vez entrenado con millones y millones de frases, el modelo nos importa para quedarnos con la W . Es la matriz que va a transformar nuestras palabras a un nuevo espacio *más interesante*.



Male-Female



Verb tense



Country-Capital

Word2Vec embedding

4.1 Task Description

To measure quality of the word vectors, we define a comprehensive test set that contains five types of semantic questions, and nine types of syntactic questions. Two examples from each category are shown in Table 1. Overall, there are 8869 semantic and 10675 syntactic questions. The questions in each category were created in two steps: first, a list of similar word pairs was created manually. Then, a large list of questions is formed by connecting two word pairs. For example, we made a list of 68 large American cities and the states they belong to, and formed about 2.5K questions by picking two word pairs at random. We have included in our test set only single token words, thus multi-word entities are not present (such as *New York*).

We evaluate the overall accuracy for all question types, and for each question type separately (semantic, syntactic). Question is assumed to be correctly answered only if the closest word to the vector computed using the above method is exactly the same as the correct word in the question; synonyms are thus counted as mistakes. This also means that reaching 100% accuracy is likely to be impossible, as the current models do not have any input information about word morphology. However, we believe that usefulness of the word vectors for certain applications should be positively correlated with this accuracy metric. Further progress can be achieved by incorporating information about structure of words, especially for the syntactic questions.

Type of relationship	Word Pair 1		Word Pair 2	
Common capital city	Athens	Greece	Oslo	Norway
All capital cities	Astana	Kazakhstan	Harare	Zimbabwe
Currency	Angola	kwanza	Iran	rial
City-in-state	Chicago	Illinois	Stockton	California
Man-Woman	brother	sister	grandson	granddaughter
Adjective to adverb	apparent	apparently	rapid	rapidly
Opposite	possibly	impossibly	ethical	unethical
Comparative	great	greater	tough	tougher
Superlative	easy	easiest	lucky	luckiest
Present Participle	think	thinking	read	reading
Nationality adjective	Switzerland	Swiss	Cambodia	Cambodian
Past tense	walking	walked	swimming	swam
Plural nouns	mouse	mice	dollar	dollars
Plural verbs	work	works	speak	speaks

Word2Vec resultados

En estas tasks de captura sintáctica y semántica del lenguaje lograron llegar a accuracies por encima de 50%. Es decir, con una cuenta tan simple como $v_{capital_de_españa} = v_{roma} - v_{italia} + v_{españa}$ lograron encontrar correctamente la respuesta más de la mitad de las veces entre todas las palabras posibles del lenguaje.

Table 8: *Examples of the word pair relationships, using the best word vectors from Table 4 (Skip-gram model trained on 783M words with 300 dimensionality).*

Relationship	Example 1	Example 2	Example 3
France - Paris	Italy: Rome	Japan: Tokyo	Florida: Tallahassee
big - bigger	small: larger	cold: colder	quick: quicker
Miami - Florida	Baltimore: Maryland	Dallas: Texas	Kona: Hawaii
Einstein - scientist	Messi: midfielder	Mozart: violinist	Picasso: painter
Sarkozy - France	Berlusconi: Italy	Merkel: Germany	Koizumi: Japan
copper - Cu	zinc: Zn	gold: Au	uranium: plutonium
Berlusconi - Silvio	Sarkozy: Nicolas	Putin: Medvedev	Obama: Barack
Microsoft - Windows	Google: Android	IBM: Linux	Apple: iPhone
Microsoft - Ballmer	Google: Yahoo	IBM: McNealy	Apple: Jobs
Japan - sushi	Germany: bratwurst	France: tapas	USA: pizza

Word2Vec ejemplo

```
import gensim.downloader as api

modelo = api.load("word2vec-google-news-300")

def evaluar_analogia(modelo, palabra_a, palabra_b, palabra_c):
    """
    Evalúa la analogía: A es a B lo que C es a ... ?
    Matemáticamente calcula:  $V_B - V_A + V_C$ 
    En la API de Gensim,  $V_B$  y  $V_C$  suman (positive), y  $V_A$  resta (negative).
    """
    print(f"\n[?] {palabra_a.upper()} es a {palabra_b.upper()} como {palabra_c.upper()} es a...")

    try:
        resultados = modelo.most_similar(positive=[palabra_b, palabra_c], negative=[palabra_a], topn=1)

        for i, (prediccion, similitud) in enumerate(resultados, 1):
            print(f"    {i}. {prediccion} (Coseno: {similitud:.3f})")

    except KeyError as e:
        print(f"    [Error] La palabra {e} no está en el vocabulario del modelo.")

# 1. Relaciones Geográficas (País -> Capital)
evaluar_analogia(modelo, 'france', 'paris', 'spain')

# 2. Relaciones Semánticas Clásicas (Género/Títulos)
evaluar_analogia(modelo, 'man', 'king', 'woman')
evaluar_analogia(modelo, 'boy', 'father', 'girl')

# 3. Relaciones Sintácticas (Tiempos verbales y plurales)
evaluar_analogia(modelo, 'walk', 'walked', 'swim')
evaluar_analogia(modelo, 'mouse', 'mice', 'child')

# 4. Dominios Específicos y Categorías
evaluar_analogia(modelo, 'fish', 'salmon', 'meat')
```

[?] FRANCE es a PARIS como SPAIN es a...
1. madrid (Coseno: 0.530)

[?] MAN es a KING como WOMAN es a...
1. queen (Coseno: 0.712)

[?] BOY es a FATHER como GIRL es a...
1. mother (Coseno: 0.831)

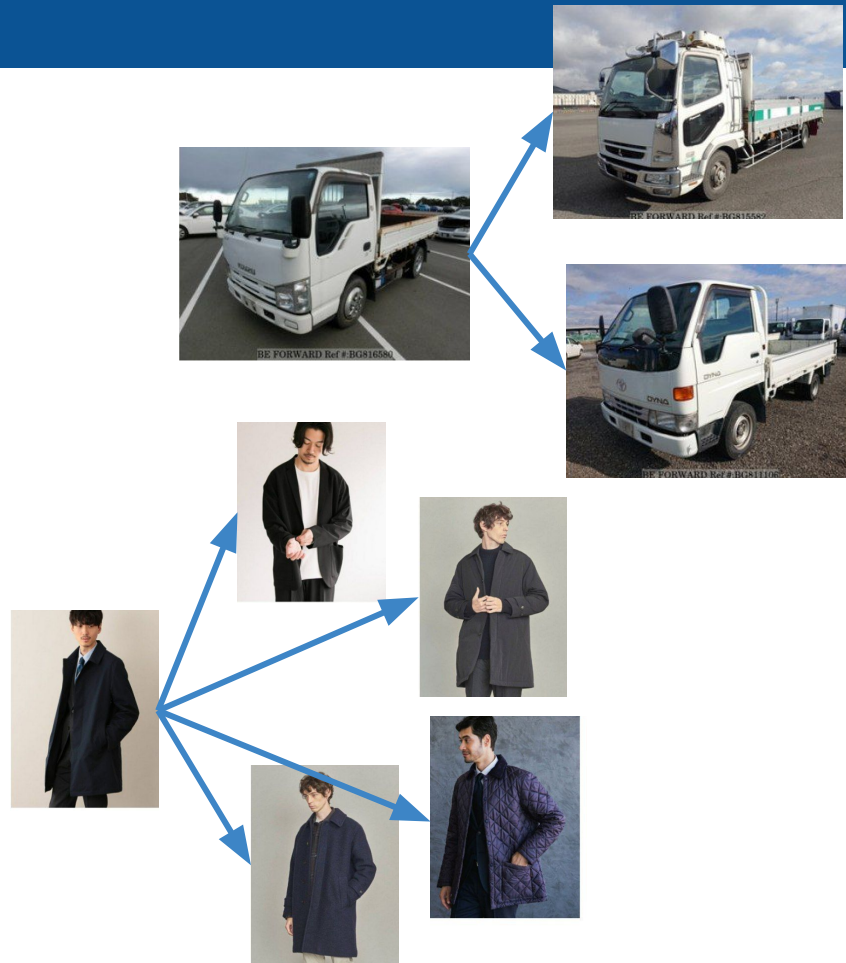
[?] WALK es a WALKED como SWIM es a...
1. swam (Coseno: 0.791)

[?] MOUSE es a MICE como CHILD es a...
1. children (Coseno: 0.625)

[?] FISH es a SALMON como MEAT es a...
1. beef (Coseno: 0.654)

La idea de *crear un embedding* en base a la secuencia de uso de ciertos objetos **no tiene por qué ser exclusivamente para el texto.**

Si nuestras *palabras* son productos que un usuario navegó en orden, podemos **usar el modelo de CBOW para crear un embedding de productos.**



Embeddings

La conclusión interesante de Word2Vec: los embeddings sirven y son realizables. Es decir, es posible aprender representaciones de las palabras en vectores con semántica utilizable.

Adicionalmente, estos vectores ya son representaciones numéricas de las palabras que podemos utilizar como input en una Red Neuronal.

Word2Vec y GloVe fueron posiblemente los embeddings más importantes en los primeros años. Hoy por hoy los embeddings siguen siendo muy importantes porque son la primera capa de las redes que procesan texto. Una capa *aprendible* que transforma el OHE de los tokens en vectores numéricos.

Language models

EL problema actual de NLP son los Language Models.

Los modelos de lenguaje buscan predecir la siguiente palabra en una secuencia de palabras en un lenguaje dado. Lo que hacen es dar una probabilidad de cuál es la palabra que continúa la frase entre todas las posibles palabras del vocabulario.

Básicamente podemos reducir el problema a uno de clasificación, en donde todas las clases posibles son todas las palabras (o tokens en realidad) existentes en mi lenguaje.

Desde que las Redes Neuronales irrumpieron en el ámbito académico, se las intentó utilizar para generar Language Models.

Las Redes Neuronales son un **formalismo matemático** poderoso pero restrictivo. La cantidad de inputs posible está fijado en la arquitectura de la red.

Cuando hacemos un MLP para resolver un problema tabular, ¿cuántas columnas le paso a la red? Todas. Siempre le puedo dar tantas columnas como tenga en mi dataset.

Cuando hacemos una CNN para resolver un problema de imágenes, ¿cuántos datos le paso a la red? Todos los píxeles. La cantidad está fijada de antemano e incluso tengo que escalar las imágenes para que *entren* en la red.

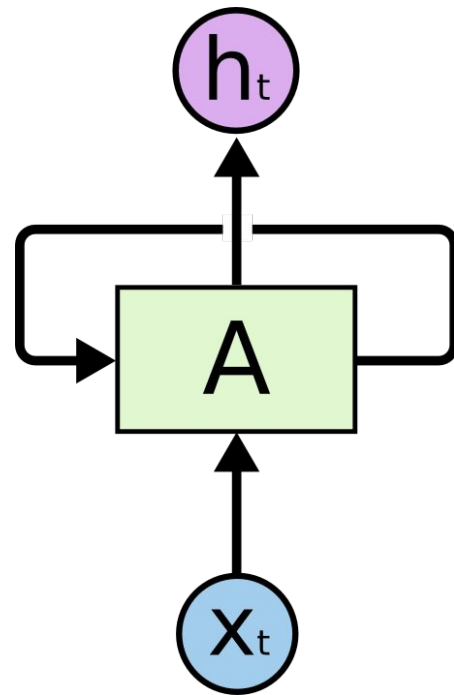
Cuando quiera hacer una Red Neuronal para texto, ¿cuántos datos le paso? ¿Las últimas 3 palabras? ¿5? ¿20? ¿Tiene que estar fijo?

Hubo intentos de moderado éxito de *fix-window* NNs, pero para evitar estas preguntas vamos a avanzar directamente hacia las Redes Neuronales Recurrentes.

En todas las redes que vimos hasta ahora las conexiones van *hacia adelante*, conectan el output de una capa con el input de la siguiente. **Esto no tiene por qué ser así.**

Las Redes Neuronales Recurrentes **rompen esta secuencialidad** reinsertando la salida de un bloque como entrada de sí mismo.

La salida del bloque (h_t en el diagrama) vuelve a entrar al bloque.



Este bloque *recurrente* ahora puede ser utilizado con secuencias de tamaño variable, simplemente vamos pasando una por una las palabras de la frase por la red. Como la salida de un paso influencia los cálculos de la siguiente pasada, esperamos que la red aproveche para *guardar* información contextual de las palabras que vió hasta ahora. De esta manera, cuando vea la cuarta palabra de una frase, la salida de la red (la quinta palabra) va a depender no solo de la cuarta palabra sino del estado interno de la red al incorporar la cuarta palabra.

Ese estado interno (se suele llamar *hidden state*) se generó por procesar las 3 primeras palabras de mi frase.

RNN Forward Pass

Un bloque recurrente lo que hace es combinar el vector de entrada x con el vector del estado oculto h_{t-1} para conseguir la nueva salida (que será el estado oculto de la siguiente pasada).

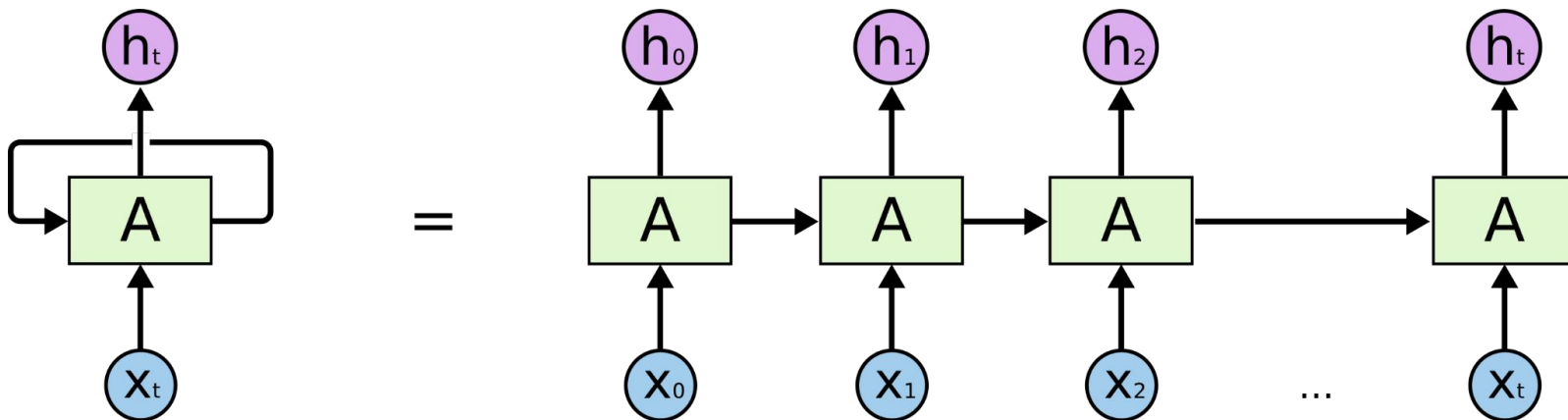
$$h_t = \tanh(W_{hh}h_{t-1} + W_{hx}x_t + b_h)$$

En cada pasada la salida generada solamente la queremos para ser el nuevo estado oculto de la siguiente pasada, excepto cuando llegamos a la última palabra de nuestra secuencia. Ahí queremos predecir la probabilidad de la siguiente palabra por lo que generamos la salida a partir del estado oculto y luego la pasamos por una *softmax*.

$$\hat{y}_t = \text{softmax}(W_{hy}h_t + b_y)$$

Unrolled RNN

Si bien el diagrama puede parecer muy distinto a lo que veníamos utilizando, en realidad podemos pensar la versión *desenrollada* del *forward pass* como se ve en el diagrama.



Ejemplo RNN

```
import torch
import torch.nn as nn
import torch.nn.functional as F

palabra = "recursivo"
letras = list(set(palabra))
vocab_size = len(letras)

char_to_idx = {c: i for i, c in enumerate(letras)}
idx_to_char = {i: c for i, c in enumerate(letras)}

x_indices = [char_to_idx[c] for c in palabra]
y_indices = [char_to_idx[c] for c in palabra[1:] + palabra[0]]

X = F.one_hot(torch.tensor(x_indices), num_classes=vocab_size).float()
X = X.unsqueeze(1)
Y = torch.tensor(y_indices)

class ModeloRNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        self.hidden_size = hidden_size
        self.rnn = nn.RNN(input_size, hidden_size, batch_first=False)
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x, h):
        out, h = self.rnn(x, h)
        predicciones = self.fc(out.squeeze(1))
        return predicciones, h
```

```
hidden_size = 12
modelo = ModeloRNN(input_size=vocab_size, hidden_size=hidden_size, output_size=vocab_size)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(modelo.parameters(), lr=0.05)

for epoch in range(1, 101):
    h0 = torch.zeros(1, 1, hidden_size)

    optimizer.zero_grad()
    predicciones, h_final = modelo(X, h0)
    loss = criterion(predicciones, Y)
    loss.backward()
    optimizer.step()

    if epoch % 20 == 0:
        print(f"Época {epoch} | Loss: {loss.item():.4f}")
```

Letra inicial: 'r'
Secuencia generada por la RNN: 'recursivo'

Letra inicial: 'e'
Secuencia generada por la RNN: 'ecursivor'

Letra inicial: 'c'
Secuencia generada por la RNN: 'cursivors'

Backpropagation Through Time: What It Does and How to Do It

PAUL J. WERBOS

A diferencia de lo que nos sucedió al introducir las CNNs, en las RNNs el algoritmo de backpropagation sí parece ser algo diferente dado que la red ya no va solo *para adelante*. Tenemos una dependencia en ciclo que cambia un poco las fórmulas.

Este algoritmo se lo denomina Backpropagation Through Time y es una simple adaptación del algoritmo original.

Al realizar las derivadas de la función que definimos en el *forward pass*, nos encontramos con las siguientes expresiones.

$$\frac{\partial L_T}{\partial W_{hh}} = \sum_{k=1}^T \frac{\partial L_T}{\partial \hat{y}_T} \frac{\partial \hat{y}_T}{\partial h_T} \frac{\partial h_T}{\partial h_k} \frac{\partial h_k}{\partial W_{hh}}$$

$$\frac{\partial h_T}{\partial h_k} = \prod_{i=k+1}^T \frac{\partial h_i}{\partial h_{i-1}}$$

Más allá de la derivación completa de las cuentas lo que sucede es que, al tener varios instantes t pero una sola matriz de pesos W_{hh} , los gradientes terminan dependiendo de la exponenciación de esa matriz.

En la práctica, si intentamos entrenar una RNN con secuencias de tamaño 25 o 30, inevitablemente los gradientes van a desvanecerse o explotar.

Problemas de las RNNs

En teoría la definición de las RNNs permite que se aprendan dependencias arbitrariamente largas en un texto. En la práctica, la multiplicación sucesiva de los gradientes hace que no sea factible aprender dependencias que vayan más allá de algunas palabras.

“Tenía que comprar las entradas. Me logueé y compré las ...” ✓

“Tenía que comprar las entradas. Me logueé pero primero me pidió cambiar la contraseña. Recién ahí pude acceder y compré las ...” ✗